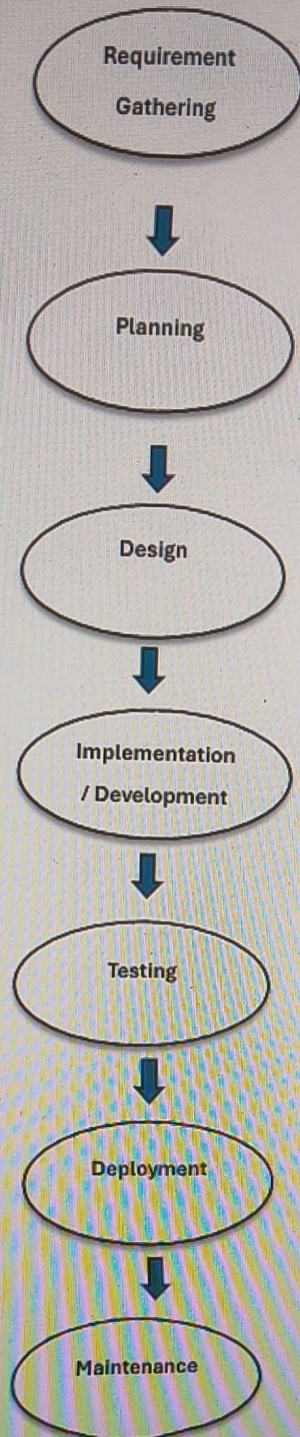


ACTIVITY - I

SDLC FLOW CHART WITH EACH PHASES

Flow Chart:



Requirement Gathering:

- Gather detailed user needs and expectations.
- Create a detailed requirements specification document outlining functional and non-functional requirements.
- Analyse and prioritize requirements for clarity.
- Review requirements with clients to ensure accuracy and completeness.

Planning:

- **Define Project Scope:** Identify the objectives, deliverables, and boundaries of the project.
- **Feasibility Study:** Assess technical, operational, and financial feasibility to determine if the project is viable.
- **Resource Allocation:** Determine the resources (human, technical, financial) required for the project.

Design:

- Define the overall architecture of the system, including hardware and software components.
- Create wireframes and prototypes for the user interface to enhance user experience.
- Design user interfaces, databases and components.
- Document technical specifications for developers to follow during implementation.

Implementation/Development:

- Write code according to the design specifications.
- Develop front-end, back-end and middleware.
- Use version control systems for collaboration.
- Document the code and development process for future reference and maintenance.

Testing:

- Develop a testing strategy and plan, including types of testing to be performed.
- Create test cases based on requirements to validate functionality.
- Execute tests (unit, integration, system, acceptance) to identify defects.
- Generate reports on testing outcomes, including pass/fail rates and defect summaries.
- Fix defects and retest iteratively.

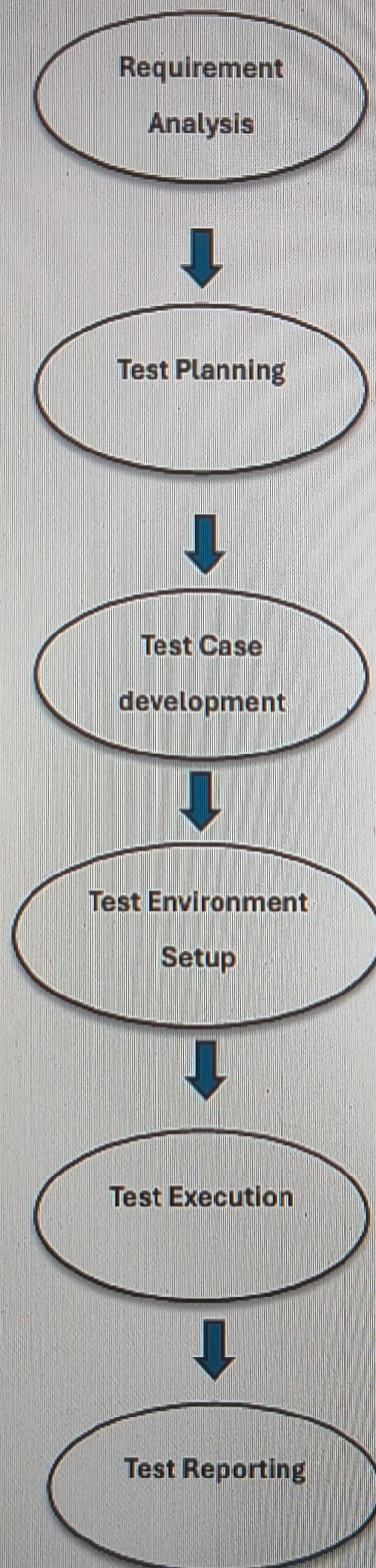
Deployment:

- Develop a deployment plan that outlines the steps for releasing the software.
- Prepare the production environment, including servers, databases, and configurations.
- Execute the deployment plan and launch the software for end-users.

ACTIVITY – II

STLC FLOW CHART WITH EACH PHASE

Flow Chart:



Requirement Analysis:

- Review and analyze the requirements to understand what needs to be tested.
- Determine which requirements are testable and which are not.
- Identify required testing types (functional, non-functional, etc.).
- Create a requirements traceability matrix to map requirements to corresponding test cases.

Test Planning:

- Establish clear objectives for the testing process, including what will be achieved.
- Choose the appropriate testing types (manual, automated, performance, etc.) based on project needs.
- Document the overall test plan, including scope, approach, schedule, and deliverables.
- Identify potential risks that could impact testing and develop mitigation strategies.

Test Case Development:

- Create detailed test cases based on the requirements and test plan.
- Identify and prepare the necessary test data required for executing test cases.
- Review and update test cases after peer review.

Test Environment Setup

- Set up hardware, software, and network required for testing.
- Install the application build in the test environment.
- Configure necessary test tools (automation, defect tracking, etc.).
- Validate that the environment works correctly.

Test Execution

- Execute test cases one by one.
- Compare actual results with expected results.
- Log defects for failed test cases.
- Re-test after developer fixes defects.

Test Reporting (Test Cycle Closure)

- Prepare a test summary report.
- Measure testing metrics (passed, failed, blocked test cases).
- Analyze defect trends and quality of the product.
- Identify lessons learned and areas for improvement.
- Get sign-off that testing is completed.

ACTIVITY – III

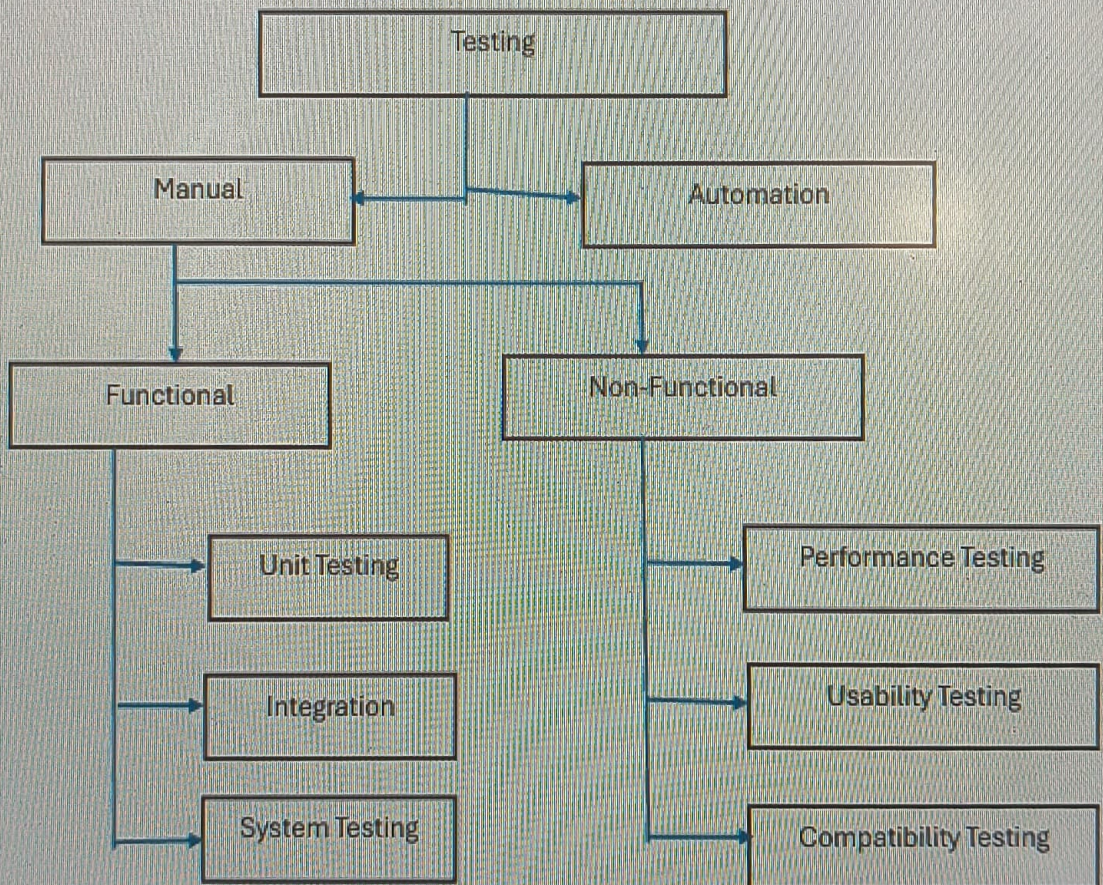
TYPES OF TESTING

Software Testing:

- Testing is the process of checking whether software works correctly.
- It helps find defects or issues before the product reaches users.
- Ensures the software meets business requirements.
- Improves the quality and performance of the application.
- Reduces risks and avoids failures after release.
- Helps deliver a reliable and user-friendly product.

Types of Testing:

- Manual Testing
- Automation Testing



Manual Testing:

- Testing is done by humans without using automation tools.
- Testers execute test cases manually and check results.

- Useful for exploratory, usability, and quick testing.
- Takes more time and effort, especially for large projects.
- No programming skills required.
- Best for testing where human observation is important.

Automation Testing

- Testing is performed using automation tools (like Selenium, Cypress, etc.).
- Faster than manual testing, especially for repeated tests.
- Requires programming or scripting knowledge.
- Useful for regression, performance, and large-scale testing.
- More accurate because human errors are reduced.
- Saves time in long-term projects.

Functional Testing

- Checks what the software is supposed to do.
- Tests all functions like login, signup, buttons, forms, etc.
- Based on business requirements.
- Ensures correct output for given inputs.
- Includes smoke, regression, integration, system, UAT, etc.
- Focus is on user actions and system behavior.

Unit Testing

- Testing individual components or functions.
- Mostly done by developers.
- Ensures each unit works correctly.

Integration Testing

- Tests how different modules work together.
- Finds interface and data flow issues.
- Comes after unit testing.

System Testing

- Tests the entire application as a whole.
- Validates end-to-end functionality.
- Done in an environment similar to production.

Non-Functional Testing

- Checks how well the software works.
- Focuses on performance, speed, usability, security, etc.
- Ensures the system handles load and stress.
- Improves user experience and reliability.
- Includes performance, security, compatibility, and usability testing.
- Not related to specific functionalities, but overall quality.

Performance Testing

- Checks speed, response time, and stability.
- Ensures system can handle load.
- Includes load, stress, and volume testing.

Usability Testing

- Checks user-friendliness of the application.
- Ensures easy navigation and good design.
- Done from a user's point of view.

Compatibility Testing

- Checks if the software works across browsers, OS, and devices.
- Ensures consistent behavior everywhere.
- Important for mobile and web apps.

ACTIVITY – IV

PRINCIPLES OF SOFTWARE TESTING

1. Testing Shows Presence of Defects

- The main goal of testing is to **find defects** in the software.
- Testing cannot guarantee the software is 100% bug-free.
- Even if no defects are found, it doesn't mean none exist.
- Testing increases confidence in the quality of the product.
- Bugs may still appear later in real-world usage.
- **Example:** You test a login page and find 3 defects. After fixing them, you may still find another issue later, such as "Forgot Password" not working.

2. Exhaustive Testing is Not Possible

- Testing every possible input, path, and scenario is impossible.
- There are infinite combinations (especially in complex apps).
- It consumes too much time and cost.
- Testers focus on important test cases instead of all cases.
- Prioritization is key for effective testing.
- **Example:** A calculator app can accept countless numbers. You cannot test all combinations like 1+1, 100+500, 99999×88888—so you test only representative cases.

3. Early Testing Saves Time and Cost

- Testing should start as early as the requirement phase.
- Early defect detection prevents bigger issues later.
- Fixing early-stage defects is cheaper than fixing after development.
- Saves time throughout the project lifecycle.
- Helps avoid design or requirement misunderstandings.
- **Example:** If the requirement says "Login with email only," but the developer planned "email or phone," finding this early avoids rebuild work.

4. Defect Clustering

- Most defects are found in a few modules.
- These modules may be complex or frequently modified.
- Testers focus more attention on these risky areas.
- Helps prioritize testing efforts.
- Re-testing critical modules is important.

- **Example:** In an e-commerce app, most defects might be found in the **payment module** because it has complex logic and integrations.

5. Pesticide Paradox

- Running the same test cases again and again will eventually find no new bugs.
- Software changes require test cases to be reviewed and updated.
- New test scenarios should be added frequently.
- This helps uncover hidden or new defects.
- Avoids testers becoming too predictable.
- **Example:** If you test a login page with only correct/incorrect passwords every time, you may miss new issues like "Remember Me" not working after an update.

6. Testing is Context Dependent

- Different types of software need different testing approaches.
- Testing a game is different from testing a banking app.
- Testing methods vary based on industry, risk, and technology.
- No single testing technique fits all situations.
- Test strategy depends on project context (web, mobile, embedded).
- **Example:**
 - For a **banking app**, security testing is most important.
 - For a **game**, performance and usability are priority.

7. Absence of Errors Fallacy

- A defect-free software does not guarantee user satisfaction.
- The product must meet both functional and business needs.
- Software may be technically correct but practically useless.
- Meeting user expectations is more important than just fixing bugs.
- A bug-free system can still fail if it solves the wrong problem.
- **Example:**
 - A travel booking app works perfectly but **does not show the total price**, so users get confused and stop using it.

BRD & FS DOCUMENT FOR AMAZON “YOU” PAGE

Business Requirements Document for Amazon YOU page

User Profile & Personalization

- The system should allow users to view and edit their personal profile information.
- The system should display personalized recommendations based on user behaviour.
- The system should show order history in an organized manner.
- The system should store and display saved addresses.
- The system should allow users to update payment methods.
- The system should maintain a Wishlist section for the user.
- The system should display recently viewed items.
- The system should allow users to manage Prime membership details.
- The system should display coupons and promotional offers available to the user.
- The system should allow users to view and manage subscriptions (e.g., Subscribe & Save).

Security & Settings

- The system should allow users to update password and security preferences.
- The system should allow users to manage login & device history.
- The system should allow users to configure notification settings.
- The system should support two-factor authentication setup.
- The system should let users download their account data.

Orders & Payments

- The system should allow users to track active orders.
- The system should provide access to invoice downloads.
- The system should support order cancellations and returns.
- The system should display rewards, gift card balance, and cashback.
- The system should allow EMI/purchase financing details to be viewed.

Support & Help

- The system should allow users to contact customer support directly.
- The system should provide quick links to FAQs and help topics.
- The system should allow users to raise support tickets.
- The system should display messages from Amazon support.
- The system should allow users to view service requests (repairs, warranty claims).

Preferences & Account Behaviour

- The system should allow users to select preferred language.
- The system should allow newsletter and marketing preferences.
- The system should allow personalization settings (ads, content).
- The system should show archived orders separately.
- The system should allow account closure requests.

Functional Specifications (FSDs) for Amazon "YOU" Page

User Profile & Personalization

- The system must fetch and display profile data from the user account database and enable inline editing.
- The system must analyse browsing, purchase, and Wishlist data to generate personalized recommendations.
- The system must retrieve order history sorted by date with filters (month/year/status).
- The system must retrieve stored addresses and allow CRUD operations (create/edit/delete).
- The system must display saved payment methods and securely support add/edit/remove operations.
- The system must retrieve Wishlist items and allow edits, deletions, and moving items to cart.
- The system must display a list of recently viewed products based on session and history logs.
- The system must fetch Prime membership renewal date, benefits, and payment history.
- The system must pull all available user-specific coupons, promo codes, and cashback offers.
- The system must list user subscriptions with options to pause, cancel, or modify frequency.

Security & Settings

- The system must allow password updates after validating old password and applying security rules.
- The system must show active device logins and support logout from individual devices.
- The system must allow toggling of email, SMS, and push notification preferences.
- The system must support enabling/disabling 2-factor authentication using OTP or authenticator apps.
- The system must generate downloadable account data in JSON/CSV format upon request.

Orders & Payments

- The system must fetch real-time order tracking details from the logistics API.
- The system must generate downloadable invoices in PDF for completed orders.
- The system must allow cancellations/returns by updating order status in backend services.
- The system must retrieve gift card balance, reward points, and cashback history.
- The system must display EMI plans, transaction summaries, and upcoming payments.

Support & Help

- The system must provide a chat/call/email interface to reach customer support.
- The system must display FAQ categories and searchable help topics.
- The system must submit support issues to backend systems and show ticket status.
- The system must display messages from Amazon support in chronological order.
- The system must show active and completed service requests like product repairs.

Preferences & Account Behaviour

- The system must allow users to select preferred language and instantly apply UI translation.
- The system must update newsletter and marketing preferences in user profile settings.
- The system must allow users to enable/disable personalized ads and save preferences.
- The system must move selected orders to an archived section and hide them from main view.
- The system must allow users to submit account closure requests and update status to “Pending Review.”